🕹️ Epochenhub Retro Games: Master-Guide

REFERENZ V2.0

Schlüsselfertige One-Shot-Erstellung neuer 16-Bit Geschichtslernspiele

HEIMBÜRGESCHULE KAHLA

1. WAS DU EINGEBEN MUSST (DER 4-ZEILEN ONE-SHOT PROMPT)

Dank unseres Master-Skills `snes-history-game` reicht eine minimalistische 4-Zeilen-Eingabe im Chat aus, um ein vollwertiges Geschichtsabenteuer schlüsselfertig zu bauen:

```
Erstelle ein neues 16-Bit Geschichtslernspiel nach unserem zentralen Skill `snes-history-game`:
- Thema: [z. B. Das Mittelalter: Ritter, Burg & Dorf / Napoleon & die Befreiungskriege]
- Klassenstufe: [z. B. Klasse 6 oder Klasse 8]
- Kapitelanzahl & Epochenprofil: [z. B. 8 Kapitel, Antike & Mittelalter (oder 10 Kapitel, Neuzeit & Moderne)]
- Historischer Zeitrahmen: [z. B. 1000 bis 1250 n. Chr. oder 1789 bis 1815]
(Optional: GitHub-Repo-URL & Vercel-Domain, falls vorab bekannt)
```

💡 **Optionale Zusatzwünsche:** Du kannst bei Bedarf besondere Wünsche anhängen (z. B. *„Mentor: Hildegard von Bingen“*, *„Fokus: Thüringer Burgen & Leuchtenburg“* oder eine Liste aus dem **Retro Asset Studio**).

🛡️ **Erinnerungs-Garantie:** Hast du im Initial-Prompt noch keine Kacheln/Assets genannt, fragt die KI vor Phase 1 genau **EINMAL** verbindlich nach deinen Wunsch-Assets aus dem Retro Asset Studio. So kannst du es niemals vergessen!

2. WELCHE ZUARBEIT VON DIR BENÖTIGT WIRD

Bereich	Was du tun musst	Was autonom passiert
1. Prompt absenden	Den 4-Zeilen-Prompt in den Chat einfügen.	Komplette Konzeption & Codegenerierung startet.
2. GitHub & Vercel	Ein leeres GitHub-Repo anlegen und in Vercel importieren (Dauer: 1 Min.).	Agent initialisiert, committet und pusht den Code autonom.
3. Assets (itch.io) (Empfohlen)	1–3 passende ZIPs aus deinem Cute-SCKR-Bundle in `assets_inbox/` ablegen.	Automatisches Entpacken, Kachelprüfung (16/32/48px), Resizing & Einsortieren in shared-assets.
4. Fehlende Relikte	*Gar nichts!*	**Claude-Pxl** generiert fehlende Leitfunde & Siegel autonom via Pixel-Code (.px.js).

3. WAS TECHNISCH & AUTONOM IM HINTERGRUND PASSIERT (7 PHASEN)

Der Agent arbeitet die 7 Phasen der One-Shot-Pipeline ohne Zwischenfragen durch:

PHASE 1

Didaktik & Friedensmodell

Erstellt 6–12 Kapitel mit 4-Punkte-Gating, 27-Einträge-Lexikon und friedenspädagogisches Interaktionsmodell (keine Shooter, sondern Logistik, Sanitätswesen, Quellen & Diplomatie).

PHASE 2

Dual-Layer Engine

Schicht 1: 16-Bit Phaser 3 WebGL/Canvas (pixelArt: true). Schicht 2: Gestochen scharfer HTML5/CSS Retina-Text, Dialoge & das 1-Klick A4-Hefterblatt (`window.print()`).

PHASE 3

Living-World & Tierbegleiter

A2-Boden-Engine mit Bayer-Dithering, Windhauch, Rauch-Partikel. Treuer Tierbegleiter ab Kap. 1 aktiv, streichelbar per [E] mit Herzchen und Spürsinn für Relikte (< 110px).

PHASE 4

Claude-Pxl Code-Generierung

Fehlen spezifische Leitfunde (z. B. Urkunde, Papyrus, Helm), schreibt der Agent direkt `.px.js`-Code und rendert binnen 20 ms saubere 1:1 Pixel-PNGs mit Alphamaske.

PHASE 5

Didaktische Mitmach-Stationen

Bauen [B], Handwerk/Logistik [C], Inventar [I], Phasen-Zeitstrahl [Z] und 360°-Quelleninspektor. Single-Proximity DOM-Labels (< 44px) gegen Label-Überlagerungen.

PHASE 6

Headless-Browser-Test (Puppeteer)

Nimmt autonom 7–10 Pflicht-Screenshots (Startscreen, Map, Dialog, Menüs, Hefterdruck) auf und analysiert sie visuell auf Clipping und Ästhetik vor dem Release.

PHASE 7: LIVE-DEPLOYMENT & EPOCHENHUB-EINHÄNGUNG

Automatischer Git-Commit & Push auf `main` (löst Vercel-Livebuild aus) + Einhängen in `src/config/games.config.ts` des Epochenhubs mit Kapitelanzahl, Zeitstrahl, QR-Code und Badge.

4. TOKENVERBRAUCH & EFFIZIENZ: MIT SKILL VS. OHNE SKILL

⚡ 75% bis 85% Token-Ersparnis durch Standardisierung

Kriterium	Bauen OHNE Skill (Standard-Chat)	Bauen MIT unserem Master-Skill
Token-Gesamtverbrauch	250.000 – 600.000+ Tokens	**45.000 – 75.000 Tokens (-80%)**
Chat-Interaktionen	20 bis 45 Ping-Pong-Nachrichten	**1 bis 2 Prompts (One-Shot)**
Entwicklungszeit	2 bis 5 Stunden zähes Nachbessern	**5 bis 12 Minuten vollautonom**
Fehlerrisiko / Halluzination	Hoch (unscharfer Text, kaputte Kollisionen)	Minimal (Drop-in Referenzmuster & Types)
Grafik-Generierung	Teure Diffusions-Bilder (matschige Pixel)	**Claude-Pxl:** Kostenloser lokaler Pixel-Code

Warum der Verbrauch ohne Skill explodiert: Jede Korrekturrunde („Text ist unscharf“, „Gebäude kollidieren“, „Hefterblatt druckt schwarz“) schickt die gesamte Codebasis erneut durch das Context-Window. Bei 20 Runden führt das zu gigantischen Kosten und „Context Rot“ (die KI vergisst frühere Fixes). Mit dem Skill sind alle Muster von vornherein definiert.

5. SCHNELL-CHECKLISTE FÜR DEIN NÄCHSTES SPIEL

- [ ] **1.** Passende ZIPs aus deinem itch.io-Bundle in `assets_inbox/` werfen (optional).
- [ ] **2.** Neues GitHub-Repository anlegen und in Vercel importieren (1 Minute).
- [ ] **3.** Den 4-Zeilen-Prompt mit Thema & Klasse in Antigravity absenden.
- [ ] **4.** Zurücklehnen und 10 Minuten warten, während die 7 Phasen autonom ablaufen.
- [ ] **5.** Im Epochenhub das fertige Abenteuer testen und den Schülern bereitstellen!